

main2

June 1, 2026

```
[1]: def print_board(board):
      """Print the 3x3 board in a readable format."""
      print()
      for i, row in enumerate(board):
          # Join each cell with a separator
          print(" " + " | ".join(row))
          if i < 2:
              print(" " + "-" * 9) # row divider (not after last row)
      print()

      # A sample mid-game board: X has center, O has top-right and bottom-right
      sample_board = [
          ["X", "O", "X"],
          ["O", "X", " "],
          [" ", " ", "O"]
      ]

      empty_board = [
          [" ", " ", " "],
          [" ", " ", " "],
          [" ", " ", " "]
      ]

      print("Sample board:")
      print_board(sample_board)
```

Sample board:

```
X | O | X
-----
O | X | 
-----
|   | O
```

```
[2]: def check_game_state(board):
      """
```

Returns:

```
    "X" -> X wins
    "O" -> O wins
    "D" -> Draw
    "_" -> Game not finished
    """

    # ----- Check rows -----
    for row in board:
        if row[0] != " " and row[0] == row[1] == row[2]:
            return row[0]

    # ----- Check columns -----
    for col in range(3):
        if (
            board[0][col] != " "
            and board[0][col] == board[1][col] == board[2][col]
        ):
            return board[0][col]

    # ----- Main diagonal -----
    if (
        board[0][0] != " "
        and board[0][0] == board[1][1] == board[2][2]
    ):
        return board[0][0]

    # ----- Anti-diagonal -----
    if (
        board[0][2] != " "
        and board[0][2] == board[1][1] == board[2][0]
    ):
        return board[0][2]

    # ----- Draw / Unfinished -----
    for row in board:
        for cell in row:
            if cell == " ":
                return "_"

    return "D"

print(check_game_state([
    ["X", "X", "X"],
```

```

        ["O", " ", "O"],
        [" ", " ", " "]
    ]))
    # X

    print(check_game_state([
        ["X", "O", "X"],
        ["X", "O", "O"],
        ["O", "X", "X"]
    ]))
    # D

    print(check_game_state([
        ["X", "O", " "],
        [" ", "X", " "],
        [" ", " ", "O"]
    ]))
    # _

```

X
D
-

```

[3]: # Static evaluation function
def static_eval(board):
    """
    Score the board from X's (Max's) perspective.

    Returns:
        +inf  → X has won
        -inf  → O has won
        score → (lines open for X) minus (lines open for O)
    """
    # First check for terminal wins
    state = check_game_state(board)

    if state == "X":
        return float("inf")

    if state == "O":
        return float("-inf")

    if state == "D":
        return 0

    # List all 8 possible winning lines
    lines = [

```

```

    # --- rows ---
    [board[0][0], board[0][1], board[0][2]],
    [board[1][0], board[1][1], board[1][2]],
    [board[2][0], board[2][1], board[2][2]],
    # --- columns ---
    [board[0][0], board[1][0], board[2][0]],
    [board[0][1], board[1][1], board[2][1]],
    [board[0][2], board[1][2], board[2][2]],
    # --- diagonals ---
    [board[0][0], board[1][1], board[2][2]],
    [board[0][2], board[1][1], board[2][0]],
]

# A line is "open for X" if O has no piece in it (X can still win via it)
x_open = sum(1 for line in lines if "O" not in line)
# A line is "open for O" if X has no piece in it
o_open = sum(1 for line in lines if "X" not in line)

return (x_open - o_open)

empty_board = [
    [" ", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "]
]

x_center = [
    [" ", " ", " "],
    [" ", "X", " "],
    [" ", " ", " "]
]

x_winning = [
    ["X", "X", "X"],
    ["O", "O", " "],
    [" ", " ", " "]
]

print("Score for empty board: ", static_eval(empty_board)) # 0 - perfectly
    ↳ equal
print("Score with X in center: ", static_eval(x_center)) # positive - X is
    ↳ better
print("Score when X has won: ", static_eval(x_winning)) # +inf

```

```

Score for empty board:    0
Score with X in center:   4
Score when X has won:    inf

```

```

[4]: def get_available_moves(board):
    """Return a list of empty positions as (row, col)."""
    moves = []
    for r in range(3):

```

```

        for c in range(3):
            if board[r][c] == " ":
                moves.append([r, c])
    return moves

def make_move(board, move, player):
    """Return a new board after placing player at (row, col)."""
    row = move[0]
    col = move[1]

    new_board = [row_[:] for row_ in board]
    new_board[row][col] = player
    return new_board

sample_board = [
    ["X", "O", "X"],
    ["O", "X", " "],
    [" ", " ", "O"]
]

print(get_available_moves(sample_board))

print_board( make_move(sample_board, get_available_moves(sample_board)[1] ,
    ↪ "X" ) )

```

```
[[1, 2], [2, 0], [2, 1]]
```

```

X | O | X
-----
O | X | 
-----
X |   | O

```

```

[5]: def board_to_key(board):
        return "".join(cell if cell != " " else "_" for row in board for cell in
    ↪ row)

class Node:
    def __init__(self, board, next_player, depth=0, parent_node=None):
        self.board = [row[:] for row in board]
        self.key = board_to_key(board)
        self.next_player = next_player
        self.depth = depth
        self.game_state = check_game_state(board)

```

```

        self.static_eval = static_eval(board)

        self.parent_node = parent_node
        self.immediate_children = []
        self.next_moves = get_available_moves(board)

        self.score = [None, None, None, None] # [max_score, max_move, min_score, min_move]
        self.alpha = float("-inf")
        self.beta = float("inf")

    def expand(self):
        next_player = "O" if self.next_player == "X" else "X"

        for move in self.next_moves:
            child_board = make_move(self.board, move, self.next_player)
            child = Node(
                board=child_board,
                next_player=next_player,
                depth=self.depth + 1,
                parent_node=self
            )
            self.immediate_children.append(child)

```

```

[6]: def print_tree(node, prefix="", is_last=True):
        level = getattr(node, "level", getattr(node, "depth", "?"))

        if prefix == "":
            print(node.key)
        else:
            connector = " " if is_last else "  "
            print(prefix + connector + "L" + str(level) + " K:" + str(node.key))

        child_count = len(node.immediate_children)

        for i, child in enumerate(node.immediate_children):
            last_child = (i == child_count - 1)

            if prefix == "":
                new_prefix = "  "
            else:
                new_prefix = prefix + (" " if is_last else "  ")

            print_tree(child, new_prefix, last_child)

```

```

[7]: def minimax(node, depth):
        nodes_explored = 1 # count this node

```

```

if depth == 0 or node.game_state in ("X", "O", "D"):
    return node.static_eval, None, nodes_explored

if not node.immediate_children:
    node.expand()

if node.next_player == "X":
    best_score = float("-inf")
    best_move = None

    for move, child in zip(node.next_moves, node.immediate_children):
        score, _, child_count = minimax(child, depth - 1)
        nodes_explored += child_count

        if score > best_score:
            best_score = score
            best_move = move

    return best_score, best_move, nodes_explored

else:
    best_score = float("inf")
    best_move = None

    for move, child in zip(node.next_moves, node.immediate_children):
        score, _, child_count = minimax(child, depth - 1)
        nodes_explored += child_count

        if score < best_score:
            best_score = score
            best_move = move

    return best_score, best_move, nodes_explored

```

```

[8]: root = Node(sample_board, next_player="X")
    score, move, count = minimax(root, depth=3)

    print("Minimax score:", score)
    print("Best move:", move)
    print("Nodes explored:", count)
    print_tree(root)

```

```

Minimax score: inf
Best move: [2, 0]
Nodes explored: 12
XOXOX__0
  L1 K:XOXOXX__0

```

```

    L2 K:XOXOXXO_0
        L3 K:XOXOXXOXO
    L2 K:XOXOXX_00
        L3 K:XOXOXXXO0
L1 K:XOXOX_X_0
L1 K:XOXOX__XO
    L2 K:XOXOXO_XO
        L3 K:XOXOXOXXO
    L2 K:XOXOX_OXO
        L3 K:XOXOXXOXO

```

```

[9]: def alpha_beta(node, depth, alpha=float("-inf"), beta=float("inf")):
    nodes_explored = 1 # count this node

    node.alpha = alpha
    node.beta = beta

    if depth == 0 or node.game_state in ("X", "O", "D"):
        return node.static_eval, None, nodes_explored

    if not node.immediate_children:
        node.expand()

    if node.next_player == "X":
        best_score = float("-inf")
        best_move = None

        for move, child in zip(node.next_moves, node.immediate_children):
            score, _, child_count = alpha_beta(child, depth - 1, alpha, beta)
            nodes_explored += child_count

            if score > best_score:
                best_score = score
                best_move = move

            alpha = max(alpha, best_score)

            if beta <= alpha:
                break

        node.alpha = alpha
        node.beta = beta
        return best_score, best_move, nodes_explored

    else:
        best_score = float("inf")
        best_move = None

```



```

    for move, child in zip(node.next_moves, node.immediate_children):
        score, _, child_count = alpha_beta(child, depth - 1, alpha, beta)
        nodes_explored += child_count

        if score < best_score:
            best_score = score
            best_move = move

    beta = min(beta, best_score)

    if beta <= alpha:
        break

    node.alpha = alpha
    node.beta = beta
    return best_score, best_move, nodes_explored

```

```

[10]: root = Node(sample_board, next_player="X")
score, move, count = alpha_beta(root, depth=3)

print("Alpha-beta score:", score)
print("Best move:", move)
print("Nodes explored:", count)

print_tree(root)

```

```

Alpha-beta score: inf
Best move: [2, 0]
Nodes explored: 7
XOXOX__O
  L1 K:XOXOX__O
    L2 K:XOXOXO_O
      L3 K:XOXOXOXO
    L2 K:XOXOX_OO
      L3 K:XOXOXO_O
  L1 K:XOXOX_X_O
  L1 K:XOXOX__XO

```

```

[11]: def get_human_move(board):
    while True:
        try:
            row = int(input("Row (1-3): ")) - 1
            col = int(input("Col (1-3): ")) - 1
        except ValueError:
            print("Please enter numbers from 1 to 3.")
            continue

```

```

        if row not in range(3) or col not in range(3):
            print("Move out of range. Try again.")
            continue

        if board[row][col] != " ":
            print("That cell is already taken. Try again.")
            continue

        return [row, col]

def get_agent_move(board, agent_player, depth=9):
    root = Node(board, next_player=agent_player)
    score, move, nodes = alpha_beta(root, depth)
    return move, score, nodes

def play_human_vs_agent(human_player="X", depth=9):
    agent_player = "O" if human_player == "X" else "X"

    board = [
        [" ", " ", " "],
        [" ", " ", " "],
        [" ", " ", " "]
    ]

    current_player = "X"

    print("Human:", human_player)
    print("Agent:", agent_player)
    print()

    while True:
        print_board(board)

        state = check_game_state(board)
        if state == "X":
            print("X wins!")
            break
        elif state == "O":
            print("O wins!")
            break
        elif state == "D":
            print("It's a draw!")
            break

        if current_player == human_player:

```

```

        print("Your turn.")
        move = get_human_move(board)
        board = make_move(board, move, human_player)
    else:
        print("Agent is thinking...")
        move, score, nodes = get_agent_move(board, agent_player,
        ↪depth=depth)
        print("Agent move:", move, "| score:", score, "| nodes:", nodes)
        board = make_move(board, move, agent_player)

    current_player = "0" if current_player == "X" else "X"

    print_board(board)

```

```
[12]: play_human_vs_agent(human_player="X", depth=1)
```

Human: X
Agent: 0

```

    |  |
    -----
    |  |
    -----
    |  |

```

Your turn.

Row (1-3): 2
Col (1-3): 2

```

    |  |
    -----
    | X |
    -----
    |  |

```

Agent is thinking...

Agent move: [0, 0] | score: 1 | nodes: 9

```

0 |  |
    -----
    | X |
    -----
    |  |

```

Your turn.

Row (1-3): 3
Col (1-3): 3

```
0 |  | 
-----
| X | 
-----
|  | X
```

Agent is thinking...

Agent move: [0, 2] | score: 1 | nodes: 7

```
0 |  | 0
-----
| X | 
-----
|  | X
```

Your turn.

Row (1-3): 2
Col (1-3): 2

That cell is already taken. Try again.

Row (1-3): 1
Col (1-3): 2

```
0 | X | 0
-----
| X | 
-----
|  | X
```

Agent is thinking...

Agent move: [2, 1] | score: 0 | nodes: 5

```
0 | X | 0
-----
| X | 
-----
| 0 | X
```

Your turn.

Row (1-3): 2
Col (1-3): 1

```

 0 | X | 0
-----
X | X | 
-----
   | 0 | X

```

Agent is thinking...

Agent move: [1, 2] | score: 0 | nodes: 3

```

 0 | X | 0
-----
X | X | 0
-----
   | 0 | X

```

Your turn.

Row (1-3): 3

Col (1-3): 1

```

 0 | X | 0
-----
X | X | 0
-----
X | 0 | X

```

It's a draw!

```

 0 | X | 0
-----
X | X | 0
-----
X | 0 | X

```

```

[13]: board = [["X", " ", " "],
               [" ", "O", " "],
               [" ", " ", " "]]

node_board = board
node_board[0][1] = "X"

print(board)
# original board also changed

```

```

[['X', 'X', ' '], [' ', 'O', ' '], [' ', ' ', ' ']]

```

```
[14]: board = [ ["X", " ", " "],  
                [" ", "O", " "],  
                [" ", " ", " "]]  
  
node_board = [row[:] for row in board]  
node_board[0][1] = "X"  
  
print(board)  
# original board stays unchanged  
  
[['X', ' ', ' '], [' ', 'O', ' '], [' ', ' ', ' ']]
```

```
[ ]:
```